

The regular case: $S_{1/2}$'s Gaussian form and the half-integer ladder

Claude, with Daniel Murfet

2026-09-06

Abstract

Unit 10 of the grammar-paper §4 autoformalisation, formalising Example `ex:fluctuation_half` — the regular-model case $\lambda = 1/2$. Two facts: (a) the *half-integer ladder* $\partial_a^n S_{1/2}(a) = \beta^n S_{(n+1)/2}(a)$, so every half-integer fluctuation function is an a -derivative of the single function $S_{1/2}$; and (b) the *Gaussian form* $S_{1/2}(a) = \sqrt{2/\beta} e^{\beta a^2/4} \int_0^\infty e^{-(u-a\sqrt{\beta/2})^2/2} du$ from completing the square. Zero `sorry/axiom`.

1 Setting

For regular statistical models the resolution is trivial and the RLCT is $\lambda = 1/2$ on every chart, so the whole fluctuation family collapses onto $S_{1/2}$. The paper records this in `ex:fluctuation_half`: $S_{1/2}$ has an error-function closed form, and all higher $S_{(n+1)/2}$ are its a -derivatives. Both facts sit directly on the derivative ladder (unit 2) and the kernel form (unit 7), needing no new machinery.

2 The things formalised

```
theorem fluctuation_half_iteratedDeriv ( : ) (h : 0 < ) (n : ) :
  iteratedDeriv n (fun a => fluctuation (1 / 2) a)
    = fun a => ^ n * fluctuation ((n : ) + 1) / 2) a

theorem fluctuation_half_gaussian ( a : ) (h : 0 < ) :
  fluctuation (1 / 2) a
    = Real.sqrt (2 / ) * Real.exp ( * a ^ 2 / 4)
      * u in Set.Ioi 0, Real.exp (-(u - a * Real.sqrt ( / 2)) ^ 2 / 2)
```

3 Proof strategy

Half-integer ladder. Induction on n . `iteratedDeriv_succ` peels one derivative; the induction hypothesis turns the inner iterated derivative into $\beta^n S_{(n+1)/2}$, and `deriv_const_mul` then `deriv_fluctuation` (the ladder $S'_\lambda = \beta S_{\lambda+1/2}$, valid since $(n+1)/2 > 0$) advance it to $\beta^{n+1} S_{(n+2)/2}$; index arithmetic $(n+1)/2 + 1/2 = (n+2)/2$ and $\beta^n \beta = \beta^{n+1}$ close it.

Gaussian form. Start from the kernel identity of unit 7 at $\lambda = 1/2$: $S_{1/2}(a) = 2^{1/2} \beta^{-1/2} \int_0^\infty u^0 e^{-u^2/2 + a\sqrt{\beta/2}u} du$. The constant is $\sqrt{2/\beta}$ (`Real.sqrt_div'` plus $2^{1/2} = \sqrt{2}$, $\beta^{-1/2} = (\sqrt{\beta})^{-1}$). Pull the constant $e^{\beta a^2/4}$ out of the integral (`integral_const_mul` backwards) and match integrands pointwise: $u^0 = 1$, and completing the square $-u^2/2 + a\sqrt{\beta/2}u = \beta a^2/4 - (u - a\sqrt{\beta/2})^2/2$ using $(\sqrt{\beta/2})^2 = \beta/2$.

4 Roadblocks and resolutions

Almost none:

- The completed-square identity is an equality with a $\sqrt{}$ that must be squared away. `nlinarith` does not prove equalities; `linear_combination (a^2/2) * hs` (with `hs :  $\sqrt{\beta/2}^2 = \beta/2$`) is the right tool — the difference of the two sides is exactly $\frac{a^2}{2}(\sqrt{\beta/2}^2 - \beta/2)$.
- `setIntegral_congr_fun` hands an applied-lambda goal; `beta_reduce` before the rewrites.
- Mathlib has no `erf`, so the Gaussian *tail* $\int_0^\infty e^{-(u-c)^2/2} du$ is left un-evaluated; the statement is faithful to `ex:fluctuation_half` up to that final notation.

5 What was Mathlib, what was new

Mathlib supplied `iteratedDeriv_succ`, `deriv_const_mul`, `integral_const_mul`, `Real.sqrt_div'/sq_sqrt`, and `linear_combination`. New is the pairing of unit 2's ladder and unit 7's kernel form into the two regular-case statements — in particular the observation that the $\lambda = 1/2$ kernel integrand, after $u^0 = 1$ and one completed square, is a shifted Gaussian.

6 Lessons

`linear_combination` is the clean way to discharge a polynomial equality that holds only after a square-root identity: name $\sqrt{c^2} = c$ and give the coefficient, rather than reaching for `nlinarith` (which targets inequalities). And a specialised value of a general closed form (here $S_{1/2}$ from the kernel form) is often cheaper to reshape directly than to re-derive from the integral definition.

7 Follow-ups

The remaining §4 items are operator-algebraic — the ladder operators $b, b^\dagger$ with $[b, b^\dagger] = 1$, the number operator, and the log-insertion generating function — which want a Weyl-algebra framework, plus the order-derivative identity $\partial_\lambda^k S_\lambda = \int (\log t)^k t^{\lambda-1} e^{\dots}$ (differentiation under the integral in the order parameter) that underlies `lem:log_shift/prop:LogODE`.